

Software Requirements Specification (SRS)

=====

Project Title:

Advanced Tic Tac Toe with SQLite and AI

1. Introduction

1.1 Purpose:

This Software Requirements Specification (SRS) outlines the functional and non-functional requirements for the development of an advanced console-based Tic Tac Toe game that supports both multiplayer and AI-driven gameplay with persistent storage using SQLite.

1.2 Scope:

The software allows users to register, login, and play Tic Tac Toe either against another player or against an AI opponent with three levels of difficulty. Game results are saved in a database and can be viewed later.

1.3 Intended Audience:

- Developers maintaining or enhancing the system.
- End users playing the game.
- Academic reviewers or instructors evaluating the project.

1.4 Definitions:

- AI: Artificial Intelligence.

- SQLite: Lightweight embedded SQL database engine.
- GUI: Graphical User Interface (not applicable, as this is console-based).

2. Overall Description

2.1 Product Perspective:

This is a standalone console application with no external dependencies beyond the SQLite library.

2.2 Product Functions:

- Register and login users.
- Play Tic Tac Toe in two modes: PvP and PvE.
- AI opponent with Easy, Medium, and Hard strategies.
- View game history stored in a SQLite database.

2.3 User Classes and Characteristics:

- Regular Users: Can register, log in, play games, and view history.
- Developers: Can access and modify source code.

2.4 Operating Environment:

- C++11 or later.
- SQLite3 library.
- Command line interface.

2.5 Design and Implementation Constraints:

- Console-based interface only.
- Uses only local file-based SQLite database.
- Minimal password security (simple hash).

2.6 Assumptions and Dependencies:

- Users will enter valid inputs when prompted.
- SQLite is available and functional.

3. Functional Requirements

3.1 User Registration and Login

- FR1: The system shall allow users to register using a unique username and password.
- FR2: The system shall store passwords as hashed strings.
- FR3: The system shall validate users during login.

3.2 Game Modes

- FR4: The system shall allow users to select between PvP and PvE.
- FR5: The system shall support three AI difficulty levels.

3.3 Game Logic

- FR6: The system shall validate all user and AI moves.
- FR7: The system shall detect game outcomes (win, draw).

3.4 History Tracking

- FR8: The system shall store game results in a database.
- FR9: The system shall allow users to view their past game results.

4. Non-Functional Requirements

4.1 Performance:

- The game shall respond to user inputs in less than 1 second.

4.2 Usability:

- The system shall be easy to use via keyboard inputs.

4.3 Reliability:

- The application shall not crash under normal operation.

4.4 Portability:

- The system shall run on any OS with a supported C++ compiler and SQLite.

4.5 Maintainability:

- The source code shall be modular and commented.

5. Appendices

A. Database Tables:

- Users(username, password)
- History(username, result, date)

B. External Libraries:

- sqlite3

C. Authors and Contributors:

- Developed by Abdullah Ahmed